

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 3

BOOSTING METHODS

Reading Time	25 minutes
Prior Knowledge	Classification, Decision Trees
Keywords	Sampling with replacement, Performance, Amount of Say

The previous lesson covered ensemble learning methods such as bagging and stacking and also applied these methods to a problem in finance. This lesson will cover another ensemble learning method called adaptive boosting in detail.

1. Introduction

Boosting was briefly discussed in Module 3, Lesson 3, and we will add to that in this lesson. Boosting, like a random forest, uses many weak learners to improve the end result. Remember that a weak learner is a learner that performs slightly better than a no-skill model. The difference with boosting and a random forest model is that the weak learners are **not** independent. These learners are built sequentially, i.e., a weak learner "improves" upon the learner before it. This improvement happens with the next learner focusing on the previous weak learner's errors. This is the fundamental concept of boosting. This lesson will introduce the reader to *adaptive* boosting.

2. Adaptive Boosting

Adaptive boosting or AdaBoost is a popular algorithm and was among the earliest boosting models introduced. The sequential development of this model occurs by adjusting the weights or importance of the previous learner's predictions. Higher weights are given to predictions that deviate from the actual value and lower weights to the predictions that are close to the actual value. In other words, more weighting is assigned to incorrect predictions, or errors are corrected iteratively. AdaBoost can work on both classification and regression problems. One should keep in mind that AdaBoost is not restricted to tree-based algorithms; however, the weak learners or base learners are homogeneous in that they must all be of the same family or class of models. As with random forests, the majority vote is the final decision. However, with AdaBoost, not all base learners have equal weight or "say" on the vote. We will use the Taiwan Credit Card default dataset to explain how AdaBoost works.

```
In [ ]: # import any libraries needed to create dataset
import random
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn import tree

# from sklearn.metrics import confusion_matrix
from sklearn.tree import DecisionTreeClassifier # , `DecisionTreeRegressor`

warnings.filterwarnings("ignore")
```

The dataset is used to identify default payments for credit cards in Taiwan. The Target class is binary such that 1 refers to a client that defaults on their credit repayments and 0 otherwise. There are 22 predictors in this dataset. More information on this dataset can be found at <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients> . We import the dataset below.

```
In [ ]: data4lesson = pd.read_excel("Copy of default of credit card clients.xlsx")
```

```
In [ ]: # Make ID column the index
data4lesson.set_index("ID", inplace=True, drop=True)
data4lesson.head()
```

```
In [ ]: # Rename the target column to 'Class'
data4lesson.rename(columns={"default payment next month": "Class"}, inplace=True)

# keep predictors in a separate list
feats = list(data4lesson.columns[0 : data4lesson.shape[1] - 1])
```

Check the dimension of the data and the frequency of each target class.

```
In [ ]: print("Rows {} columns {}".format(data4lesson.shape[0], data4lesson.shape[1]))
data4lesson["Class"].value_counts()
```

Let's begin the steps for AdaBoost.

2.1 Step 1: Sample Weights

The first step for AdaBoost is to perform random sampling with replacement on our dataset. To do this, we initially assign each observation an equal weight, i.e., $1/N$ for N being the number of observations. To explain the process of random sampling, we will insert a cumulative column of the weights called `RangeSelect` to illustrate the idea.

```
In [ ]: # Assign equal weights to all samples
N = len(data4lesson)
data4lesson["probR1"] = 1 / (1.0 * N)
data4lesson["RangeSelect"] = data4lesson["probR1"].cumsum()
# Show last 5 columns
data4lesson[["PAY_AMT5", "PAY_AMT6", "Class", "probR1", "RangeSelect"]].head()
```

Now that we have the cumulative column, we are able to explain the method of sampling.

2.2 Step 2: Sampling with Replacement

Think of each value in the `RangeSelect` column as an upper limit with the lower limit being the value in the observation, or row, before it. For example, the output above ID 2 would have a `RangeSelect` of $[\text{0.000033}, \text{0.000067})$. Now draw a random number in the range $[0, 1]$. Suppose this number is 0.000043 . We therefore add ID 2 to our sample since the random number of 0.000043 is in the range $[\text{0.000033}, \text{0.000067})$, which corresponds to ID 2. If our random number drawn was 0.00014 , then ID 5 would be added to the sample since 0.00014 is in the range $[\text{0.000133}, \text{0.000167})$. One may notice that observations with large ranges would have a higher chance of being selected. Hence, large weights or `probR1` values increase the chances of being sampled, which is crucial for the AdaBoost algorithm. Fortunately, Python has a more elegant and efficient way of performing random sampling with replacement, so we won't need the `RangeSelect` column anymore.

```
In [ ]: # Drop `RangeSelect` column
data4lesson.drop("RangeSelect", axis=1, inplace=True)
```

We'll use the pandas `sample` function to sample with replacement based on the weights column `probR1`. The amount of samples is the same as N , that is, the same number of observations as our original dataset. There will be duplicate observations in this new dataset since the sampling is done with replacement.

```
In [ ]: # Random selection based on weights
# specify seed in case we wish to replicate
# the Random Sampling with replacement
random.seed(10)

data4lesson1 = data4lesson.sample(
    len(data4lesson), replace=True, weights=data4lesson["probR1"]
)
```

2.3 Step 3: Fit a Base Learner

The next step in AdaBoost is to fit a weak classifier or base learner to the samples. For this example, we will use decision trees as our base learners. We'll call this learner `weak_l1` and fit it to the samples.

```
In [ ]: # Since it is a stump the max depth is 1
weak_l1 = DecisionTreeClassifier(random_state=10, max_depth=1)

X = data4lesson1.loc[:, feats]
y = data4lesson1["Class"]

weak_l1.fit(X, y)
```

We should be able to view the flow diagram of the stump.

```
In [ ]: # visualize the stump
# features we used

fn = feats

# labels of the target class

cn = ["0", "1"]

fig = plt.figure(figsize=(25, 20))
_ = tree.plot_tree(weak_l1, feature_names=fn, class_names=cn, filled=True)

# save the figure to file

# fig.savefig('imagenam.png')
fig.savefig("decistion_tree.png")
```

Figure 1: The Decision Stump of the Base Learner at the First Iteration

Notice the base learner found *Pay_0* to be the strongest predictor to split on. Let us compare the predictions to the actual values.

```
In [ ]: data4lesson1["pred1"] = weak_l1.predict(X)
```

```
In [ ]: data4lesson1[["Class", "pred1", "probR1"]].head()
```

2.4 Step 4: Calculate Performance

The snapshot of just the first five rows shows one misclassification for ID 16973. The number of misclassifications will determine the *amount of say* or *performance* of this classifier. The formula for the amount of say is

$$\text{Amount of say} = \frac{1}{2} \ln \left(\frac{1 - \text{Total Error}}{\text{Total Error}} \right) \quad (1.1)$$

where the **Total Error** is simply the sum of weights of misclassified observations. The Total Error for the base classifier at our first iteration, i.e., *weak_l1* is

```
In [ ]: # Total error of weak_l1
E_1 = round(
    np.sum(
        np.where(
            data4lesson1["Class"] != data4lesson1["pred1"], data4lesson1["probR1"]
        )
    ),
    5,
)

E_1
```

The Amount of say in (1.1) against the total error in general is shown below.

```
In [ ]: n_pts = 100
TE = 0.01 + np.arange(n_pts) * (1 - 0.01) / n_pts
AoS = 0.5 * np.log((1 - TE) / TE)
plt.plot(TE, AoS)
plt.ylabel("Amount of Say")
plt.xlabel("Total Error")
```

Figure 2: Performance or Amount of Say against Total Error for a Classifier

From figure 2, we see that for close to perfect classifications, the base classifier will have a performance that tends to infinity. For almost all incorrect classifications, the learner will have performance that tends to negative infinity. For a Total Error of 0.5, this results in a performance of zero. If all samples had equal weights, this would be equivalent to a no-skill classifier. In practice, a small error term is added to the Total Error to avoid the case when the Total Error is zero. Our first base learner has a Total Error of 0.1796, which results in a performance of 0.7596. Remember that AdaBoost focuses on the misclassifications by adjusting the weights in an iterative manner. Let us now look into this process of adjusting the weights.

For the observations that were misclassified, the formula to adjust the weights is given by,

$$\text{Adjusted weight} = \text{Current Weight} \times e^{\text{performance}} \tag{1.2}$$

similarly the formula for adjusting the weights of the correct classification is,

$$\text{Adjusted weight} = \text{Current Weight} \times e^{-\text{performance}} \tag{1.3}$$

Notice the difference between (1.2) and (1.3) is the sign of the exponent. This implies that correct classifications are downweighted whereas misclassifications are adjusted to larger weights. Applying (1.2) and (1.3) to the weights creates the new weights shown below.

```
In [ ]: # performance of weak_l1
p1 = 0.5 * np.log((1 - E_1) / E_1)
# Call the new weights `probR2` since it is the second iteration
data4lesson1["probR2"] = np.where(
    data4lesson1["Class"] != data4lesson1["pred1"],
    data4lesson1["probR1"] * np.exp(p1), # misclassifications (1.2)
    data4lesson1["probR1"] * np.exp(-p1),
)
```

```
In [ ]: data4lesson1[["Class", "pred1", "probR1", "probR2"]].head()
```

We can see the correct classifications have new weights that are lower than the original weights of 3.3×10^{-5} whereas the misclassifications have weights that have increased. Remember that an increase in weight would increase the chances of choosing that observation when sampling with replacement. One thing to take note of is that the new weights are normalized such that they sum to 1 as were the original weights in column `probR1`.

```
In [ ]: # normalize the weights
data4lesson1["probR2"] = data4lesson1["probR2"] / data4lesson1["probR2"].sum()
```

This brings us back to Step 1 with a new set of weights to repeat steps 2 to 4. The number of times we would repeat these cycles would depend on the number of weak classifiers we specify. This is the iterative process that AdaBoost follows to develop the final model. Ideally, we would want to continue this process until all observations are classified correctly, but this can allow for overfitting. Hyperparameter tuning should be taken into consideration. To illustrate the final prediction of the model, let's assume we had chosen 5 base learners and they each had the following performance or Amount of Say and prediction.

```
In [ ]: dat = [[0.7596, 1], [0.7821, 1], [0.79019, 1], [0.8241, 0], [0.8410, 0]]
model_summary = pd.DataFrame(data=dat, columns=["performance", "prediction"])
model_summary.index.name = "iteration"
model_summary
```

The hypothetical result above over 5 iterations shows 3 base learners predicting class 1 and 2 base learners predicting class 0. The final vote is based on the sum of performance for each class voted, i.e., instead of the counts being used, the sum of the performance is used. The calculation below shows that class 1 would be favored since the sum of the performance for base learners in favor of that class is 2.332, which is greater than the sum of the performance for base learners in favor of class 0.

```
In [ ]: class_0 = np.sum(
    np.where(model_summary["prediction"] == 0, model_summary["performance"], 0)
)
class_1 = np.sum(
    np.where(model_summary["prediction"] == 1, model_summary["performance"], 0)
)

print("Weight of Votes ")
print("Class 0: {} Class 1: {}".format(class_0, class_1))
```

Something to keep in mind is that the example above showed the updating procedure on the entire dataset whereas in practice we would be using the training set to update and develop AdaBoost. We've also shown the process using weak decision stumps, but there is evidence to support more advanced learners being effective (Chuan et al. 3).

Boosting has found success in the data science field especially in Kaggle competitions. There is one more boosting technique we will cover in the next lesson called gradient boosting.

3. Conclusion

This lesson focused on adaptive boosting and showed the iterative updating procedure to derive the final model and prediction. The procedure was shown for a classification problem, namely predictive credit card payment defaults. In the next lesson, we'll look at

another boosting algorithm called gradient boosting and compare all ensemble learning methods covered in this module to a common problem.

References

1. Chuan, Yijian, et al. "The Success of AdaBoost and Its Application in Portfolio Management." *arXiv*, 2021, pp. 1–6.
2. University of California, Irvine Machine Learning Repository. "Default of Credit Card Clients." <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.